ENGINEERING REPORT

TicketWave — Code Review Methodology & Findings

An analysis of the review methodology, critical defects discovered, verified fixes, and automated regression gates.

SECTION 1

Review Methodology & Provenance

Auditing the system through interactive, narrow, and concrete review passes.

Two-Step Review Pattern

Step 1: Finding

Prompt: *"Review for best-practice violations and code smells, and list the findings ranked by severity."*

Deliberately splits finding from fixing so an explicit inventory of findings exists as an immutable artifact before code changes.

Step 2: Fixing

Prompt: *"Apply the critical corrections identified in the code review."*

Applies corrections based directly on the ranked finding list, preventing scope creep and ensuring targeted resolution.

Dual-Axis Auditing Strategy

Axis 1: User Stories

Vertical slice audit across DB, service logic, controller, and Angular components. Forced verdict per story: `PASS`, `FAIL`, or `PARTIAL`.

Axis 2: Architectural Layers

Horizontal audit examining database schemas, backend APIs, frontend routes, and dev ops against layer-tailored conformance scales.

Fixed Baseline

All review passes graded against a written repository standard (`CLAUDE.md`) created in Phase 0 before code was written.

System Inventory & Scope Examined

Architectural Component	Examined Volume	Focus & Verification Target
Controllers & Endpoints	20 Controllers / 69 Endpoints	OpenAPI alignment, RBAC, DTO decoupling
Services & Repositories	24 Services / 20 Repositories	Business rules, concurrency locks, data mapping
Data Models & Migrations	43 Entities / 33 Liquibase Logs	FK constraints, precision/scale, Liquibase-JPA drift
Backend Test Suites	100 Classes (670 Unit / 35 IT)	Line & branch coverage, concurrency verification
Frontend & E2E Tests	57 Specs (401 Tests) + 2 Playwright	Guards, interceptors, form validations, E2E flows

Severity Model & Defect Classification

4	7
Critical Findings	High Severity
6	**3**
Medium Severity	Low Severity

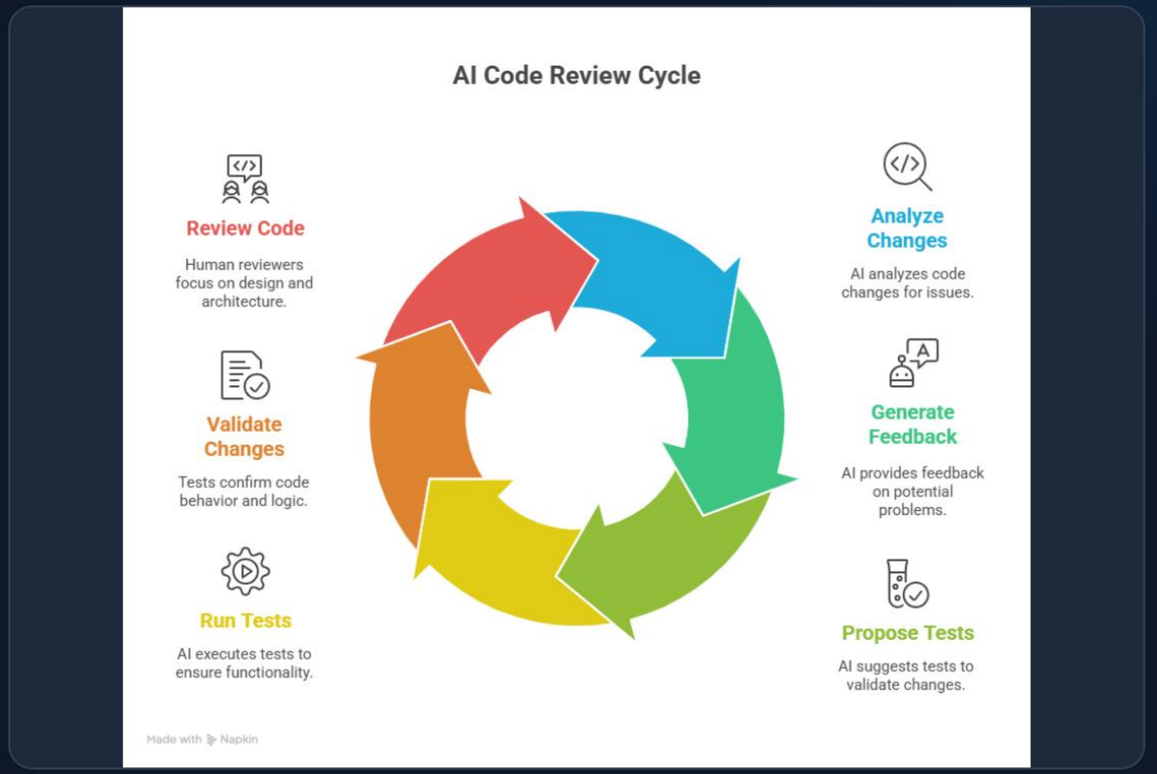
The Primary Threat: Silent Defects

The recurring theme across critical findings was **silence**: bugs that produced no exceptions, logged no errors, and passed basic tests while silently operating incorrectly.

Double-pass auditing was specifically effective at forcing these invisible logic bugs into the open.

Critical Findings & Immediate Fixes

- **Duplicate Filter Registration:** Auto-registration of `JwtAuthenticationFilter` bypassed the security chain, silently discarding auth context without error.
- **3DS Payment Result Loss:** Detached entity status with `open-in-view: false` left payments stuck in `PENDING_3DS` while returning success.
- **Disabled IT Suite:** Missing Failsafe execution meant `mvn verify` reported a green build while skipping all integration tests.



High & Medium Fixes

Concurrent Refund Race Conditions

Added `@Version` optimistic locking to `Booking` mapped to `409 CONCURRENT_UPDATE` to prevent double refunds.

Deadlock Prevention in Seat Booking

Enforced ascending ID lock acquisition order during checkout races, eliminating lock wait cycles.

Reverse Proxy Header Trust

Configured native forward headers strategy to trust `X-Forwarded-For` strictly from trusted internal proxies.

pes of Security Audits

1

mpliance Audits

2

Risk Assessment Audits

Social Engine Audits

4

nfiguration Audits

5

External Vulnerability Assessment

Penetratio Testing

Correction Verification Strategy

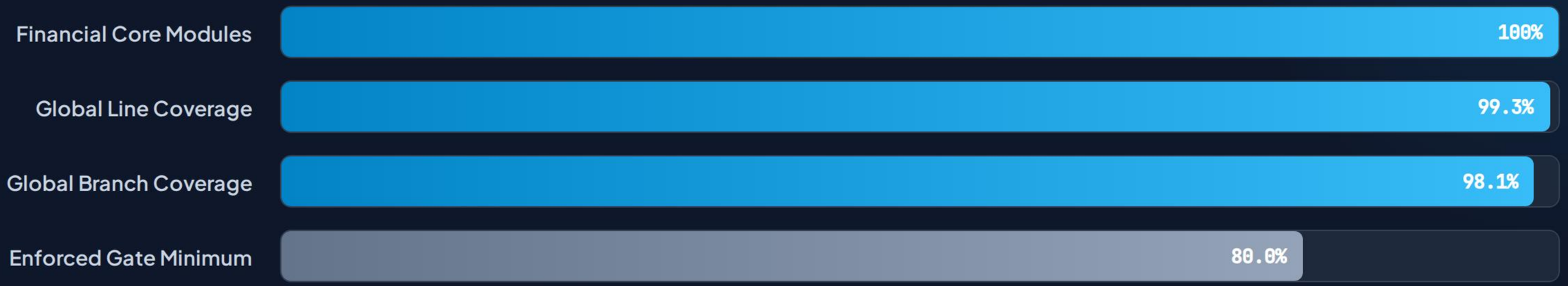
Mandatory Regression Tests: Every bug fix shipped with a dedicated test reproducing the original failure condition before verification.

Real PostgreSQL Isolation: Concurrency issues (e.g. seat locking deadlocks) were verified against real PostgreSQL instances via Testcontainers.

Isolated Fixture Generation: Moved unique identifier creation inside test helpers to guarantee sequential test suite re-runnability.

Single Command Suite Run: Complete suite reproducibility verified via `cd backend && ./mvnw clean verify`.

Automated Quality Gates & Coverage



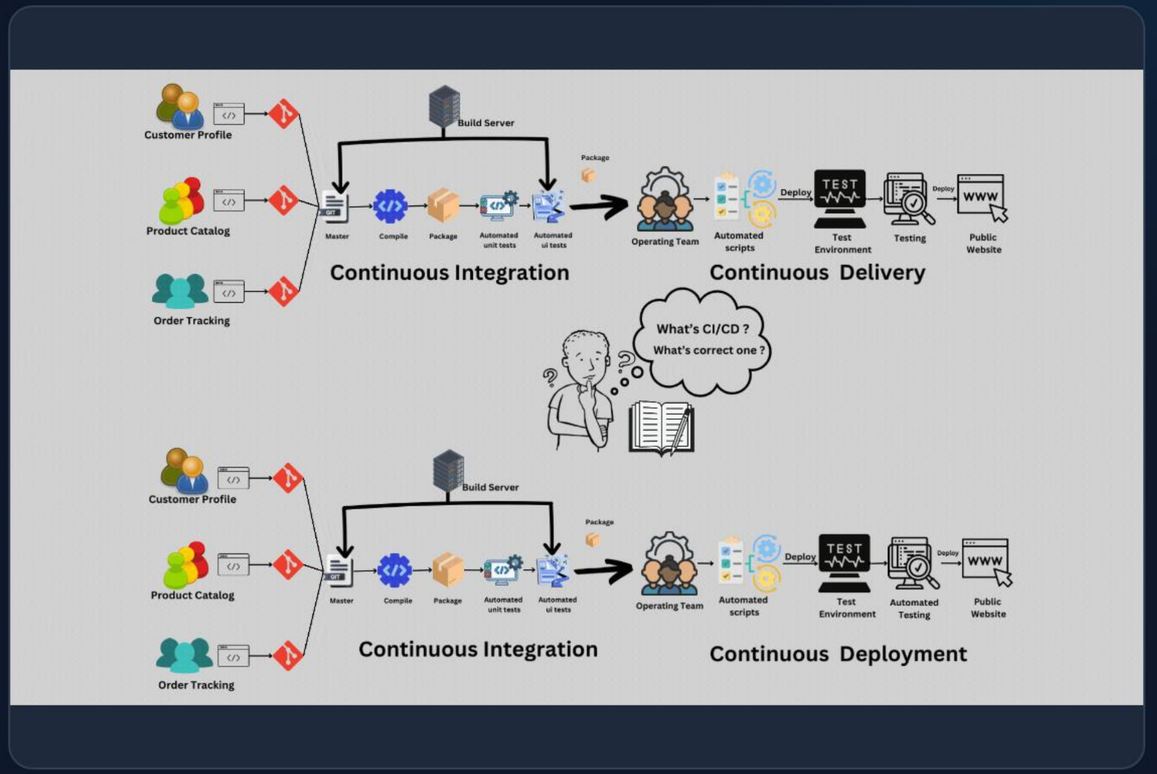
JaCoCo gates enforce minimum limits on every build. Core modules (`PricingServiceImpl` , `RefundPolicyService`) enforce strict 100% line & branch coverage.

Key Engineering Learnings

Write Standards Early: Pre-defining `CLAUDE.md` eliminated subjective arguments during reviews.

Convert Findings to Gates: Review findings were converted into CI checks so defects fail the build permanently.

Record Defect Context in Code: Comments naming the bug prevent future regressions during refactoring.



TICKETWAVE REVIEW COMPLETE

Questions & Discussion

All automated gates active in CI pipeline via GitHub Actions.

```
mvn clean verify | .github/workflows/ci.yml
```

Image Sources

https://cdn.prod.website-files.com/66601b586a17566fb54a7070/68dbae2e280b61f328fdba0a_508658e1.png

Source: www.startearly.ai

https://www.fortinet.com/content/dam/fortinet/images/cyberglossary/6-types-of-security-audits1.png

Source: www.fortinet.com

https://miro.medium.com/1*TgAGJ_EigoPBprtPXPRTqQ.gif

Source: medium.com